

# Análisis de datos

Carrera de Especialización en  
Inteligencia Artificial

Bimestre 4B2025



# Docentes



Esp. Lic. María Carina Roldán  
[macroldan@fi.uba.ar](mailto:macroldan@fi.uba.ar)

Esp. Ing. Ariadna Garmendia  
[arigarmendia@gmail.com](mailto:arigarmendia@gmail.com)



# Método de evaluación

- Para aprobar la materia deberán realizar **una exposición grupal** en la clase 8 (lo explicaremos al final de la clase).
- Se recomienda hacer avances y realizar consultas semanalmente, incorporando los conceptos vistos en cada clase.
- La presentación es obligatoria. No estar presente implica desaprobación de la materia.
- La nota final surgirá de la calidad de la exposición.

# Información útil

- 1 Material de clase: [Campus posgrado](#)
- 2 Notebooks: [Repositorio de la materia](#)



# Objetivos de la materia

1

Aprender a realizar un análisis exploratorio de datos en detalle.

Sin un analisis no se puede entrenar un modelo. Hay que entender los datos que tengo

2

Entender qué mecanismos existen para el tratamiento de datos y cuáles son recomendados para el entrenamiento en un modelo de ML.

Ahora que tengo los datos, puedo entender que hacer con ellos



# Enfoque

- 1 La materia tiene un foco fuertemente práctico.
- 2 Vemos la teoría sin profundizar en la base matemática y ni estadística. Nos apoyamos en las bases proporcionadas en las materias anteriores.

En esta materia analizamos y preparamos datos pero no entrenamos modelos.



# Programa de la materia

1

## Introducción al análisis de datos

Flujo de trabajo típico. Aplicaciones. Configuración del entorno. Introducción a las principales bibliotecas.

28/8

2

## Análisis exploratorio de datos (EDA)

Tipos de variables. EDA y visualización de variables numéricas y categóricas. Identificación de valores faltantes y outliers. Relación entre variables.

4/9

3

## Taller práctico - parte 1

Trabajo en grupos: EDA y visualización inicial de un dataset. Buenas prácticas. Puesta en común.

11/9

4

## Preprocesamiento y limpieza de datos

Presentación de grupos de trabajo final.

Prevención de data leakage. Tratamiento de datos faltantes. Tratamiento de outliers.

18/9

5

## Feature engineering

Cardinalidad, codificación, discretización. Desbalance. Creación de nuevos features. Normalización y estandarización.

25/9

6

## Reducción de dimensionalidad

Métodos de extracción y técnicas para selección de features. **Bonus:** EDA de datos no estructurados.

2/10

7

## Taller práctico - parte 2

Trabajo en grupo: preprocesamiento y feature engineering de un dataset. Buenas prácticas. Puesta en común. Consultas.

9/10



13/10 - límite para compartir el material para la exposición final.

8

## Exposición final

Exposición de trabajos finales.

**Bonus:** Herramientas para automatizar el EDA y otras tareas del flujo de trabajo.

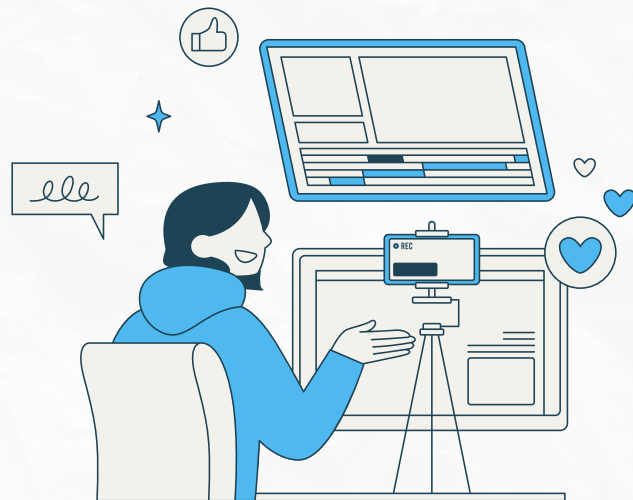
16/10

# ¿Qué es el análisis de datos?



Es el proceso de explorar, limpiar y modelar datos para extraer información útil y tomar decisiones.

Es clave para garantizar que los modelos de IA trabajen con datos confiables y adecuados para el problema.





# Tipos de análisis de datos

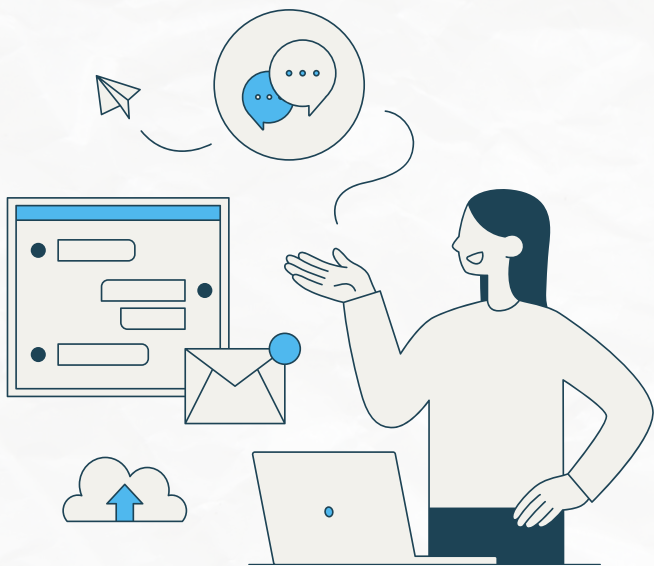
Estos tipos ser niveles, suelen aplicarse en secuencia

- 1 Descriptivo: mediante herramientas estadísticas, ayuda a explicar qué ocurrió.
- 2 Diagnóstico: determina por qué ha ocurrido algo.
- 3 Predictivo: predice resultados futuros basándose en estadísticas, modelos y aprendizaje automático.
- 4 Prescriptivo: identifica la mejor recomendación posible a seguir.  
Hacer algo en base al resultado de la predicción.  
Lo realiza alguien de negocio



**¿Qué casos de uso de análisis de datos conocen?**

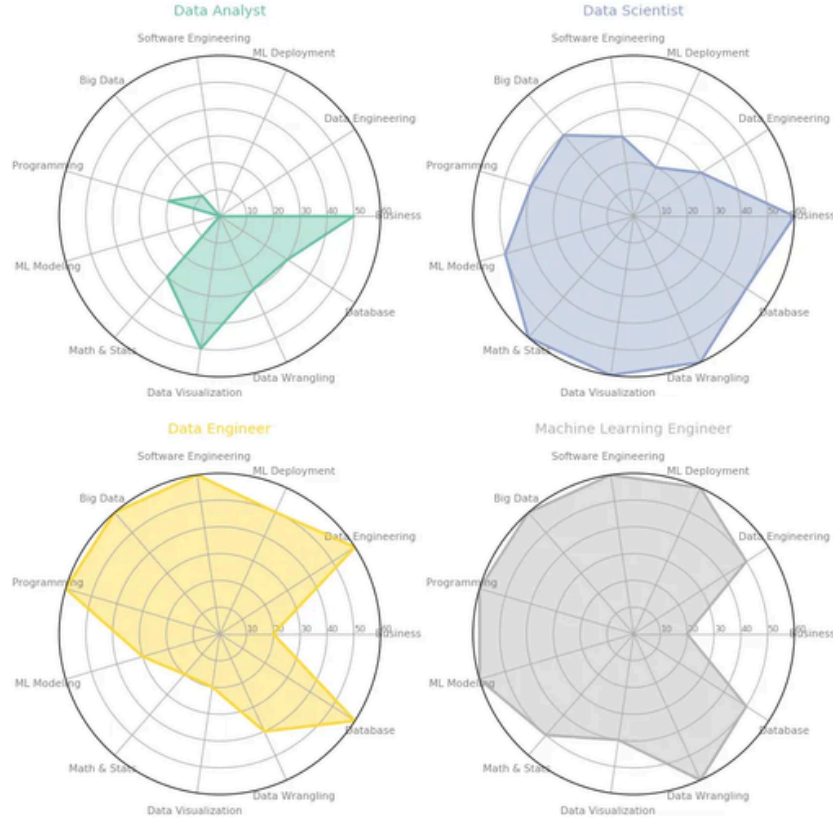
# Aplicaciones



- 1 Detección de fraudes: análisis de transacciones y detección de anomalías.
- 2 Salud: identificar tendencias en enfermedades y mejorar tratamientos.
- 3 E-commerce y retail: analizar hábitos de compra para recomendar productos.
- 4 Manufactura: identificar productos defectuosos en una línea de producción.
- 5 Redes sociales: entender opiniones de los usuarios y evaluar sus interacciones con el contenido.

¿Cuál es la diferencia entre resolver problemas mediante análisis de datos y hacerlo con ML?

# Análisis de datos vs. ML



# Flujo de trabajo del análisis de datos tradicional

1 Identificar el problema



2 Recopilar los datos.



3 Limpiar y transformar los datos



4 Analizarlos (estadística y visualización)



5 Interpretar los resultados



# 1. Identificar el problema

1

Definir el objetivo a alcanzar.

2

Caracterizar la situación actual.

3

Formular la pregunta o o hipótesis que se desea probar con el análisis.

4

Definir el alcance del problema

5

Establecer los entregables



## 2. Recopilar los datos

No siempre es fácil recopilar los datos o no alcanza, entonces hay que buscar datos externos

- 1 Definir una estrategia para recolectar y combinar los datos: ¿Qué datos se necesitan?
- 2 Identificar fuentes internas (por ejemplo del software de gestión de la empresa) o externas (de otras compañías, servicios de consultoría, de entidades gubernamentales, etc.)
- 3 Gestionar desafíos comunes: inconsistencia, errores tipográficos, falta de estandarización y valores faltantes, entre otros.



# 3. Limpiar y transformar los datos

- 1 Remover información duplicada o irrelevante. Corregir errores tipográficos e inconsistencias.
- 2 Implementar una estrategia para el manejo de datos faltantes.
- 3 Cambiar la estructura o el formato.
- 4 Remover o enmascarar PII (Personal Identifiable Information) y generalizar atributos sensibles.



Ejemplo basico: transformar mails que terminan en .con por .com

## 4. Analizar los datos

1

Aplicar herramientas estadísticas descriptivas (Ej: media, mediana) y pruebas inferenciales (t-tests, ANOVA) para descubrir patrones, tendencias y relaciones.

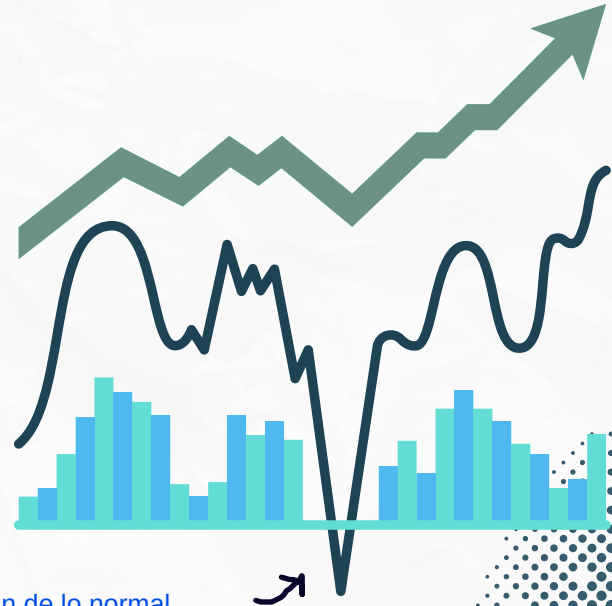
2

Crear gráficos (de barras, scatterplots) por medio de bibliotecas como Matplotlib o Seaborn para explorar los datos y comunicar resultados efectivamente.

3

Desafíos: manejar outliers engañosos, interpretar estadísticas complejas correctamente y evitar tergiversar los datos al visualizarlos.

outliers = anomalías = datos atípicos: son datos que se escapan de lo normal  
No siempre los outliers son datos invalidos, hay que saber distinguir



# 5. Interpretar los resultados

Normalmente lo realiza alguien de negocio, quien debe interpretar correctamente los resultados y decidir que hacer.



- 1 Traducir los resultados estadísticos y visualizaciones en respuestas accionables, conectándolos al problema original o hipótesis.
- 2 Evaluar el significado práctico y estadístico de los resultados (Ej: intervalo de confianza) para determinar su confiabilidad e importancia para el problema en cuestión.
- 3 Presentar los resultados a los interesados. Ajustar el mensaje a la audiencia.
- 4 Evitar generalizar, y considerar posibles sesgos y limitaciones en los datos o el análisis. Gestionar interpretaciones conflictivas o incertidumbre.

# Flujo de trabajo del análisis de datos para IA



Una vez que el modelo está en producción, también es importante el proceso iterativo ya que los datos cambian. También es importante cambiar la técnica utilizada para ver mejoras posibles.

# **¿Cómo se relaciona el flujo de trabajo con el trabajo grupal?**



# Repositorio e instalación



# Introducción a bibliotecas relevantes



Recolección de datos

BeautifulSoup

SQLAlchemy

EDA, preprocesamiento, limpieza

pandas

NumPy

Casi no se va a usar porque  
pandas lo uso por detras.

Análisis estadístico, reducción de  
dimensionalidad

scikit  
learn

SciPy

statsmodels

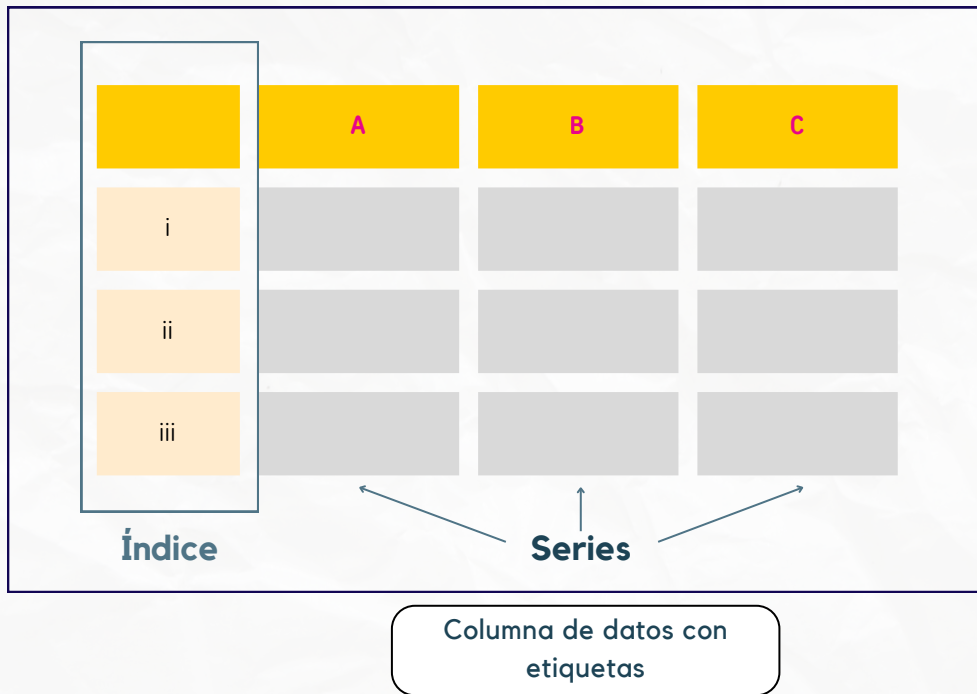
Visualización y análisis

matplotlib

seaborn



# Pandas. Terminología básica





# Pandas. Documentación

Sitio oficial

Cheatsheet

**pandas**

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

[Install pandas now!](#)

Latest version: 2.2.3

- What's new in 2.2.3
- Release date: Sep 20, 2024
- Documentation (web)
- Download source code

Follow us

Recommended books

Getting started

- Install pandas
- API reference
- Getting started

Documentation

- User guide
- API reference
- Contributing to pandas
- Release notes

Community

- About pandas
- Ask a question
- Ecosystem

With the support of:

**Data Wrangling**  
with pandas Cheat Sheet  
<http://pandas.pydata.org>

[Pandas API Reference](#) [Pandas User Guide](#)

**Creating DataFrames**

```
df = pd.DataFrame({
    "a": [4, 5, 6],
    "b": [7, 8, 9],
    "c": [10, 11, 12],
    index = [1, 2, 3]
})
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=["a", "b", "c"])
```

Specify rows for each row.

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12],
     index = pd.MultiIndex.from_tuples(
        [(1, 2), (1, 3), (2, 3)],
        names=["a", "b"])
})
```

Create DataFrame with a MultiIndex.

**Method Chaining**

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = pd.DataFrame({
    "columns": {
        "variable": "var",
        "value": "val"
    }
})
```

```
df["variable"] = "var"
df["value"] = "val"
df["value"] = "val"
df["value"] = "val"
```

**Tidy Data** - A foundation for wrangling in pandas

Tidy data complements pandas's vectorized operations. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

In a tidy data set:

- Each variable is saved in its own column
- Each observation is saved in its own row

Visual representation:  $M \times A$

**Reshaping Data** - Change layout, sorting, reindexing, renaming

**pd.melt(df)**  
Gather columns into rows.

**df.pivot(columns="var", values="val")**  
Spread rows into columns.

**pd.concat([df1, df2])**  
Append rows of DataFrames.

**pd.concat([df1, df2], axis=1)**  
Append columns of DataFrames.

**df.sort\_values("var")**  
Order rows by values of a column (low to high).

**df.sort\_values("var", ascending=False)**  
Order rows by values of a column (high to low).

**df.reset\_index()**  
Sort the index of a DataFrame.

**df.reset\_index()**  
Reset index of DataFrame to row numbers, moving index to columns.

**df.drop(columns=["Length", "Weight"])**  
Drop columns from DataFrame.

**Subset Observations - rows**

**df[df.Length > 7]**  
Extract rows that meet logical criteria.

**df.drop\_duplicates()**  
Remove duplicate rows (only considers columns).

**df.sample(frac=0.5)**  
Randomly select fraction of rows.

**df.sample(n=10)**  
Randomly select n rows.

**df.reset\_index(inplace=True)**  
Reset index to row numbers.

**df.reset\_index(inplace=True)**  
Reset index to row numbers.

**df.reset\_index(inplace=True)**  
Reset index to row numbers.

**Subset Variables - columns**

**df[["width", "length", "species"]]**  
Select multiple columns with specific names.

**df["width"] or df.width**  
Select single column with specific name.

**df.filter(regex="regex")**  
Select columns whose name matches regular expression regex.

**Using query**

**query()** allows Boolean expressions for filtering rows.

```
df.query("Length > 7")
df.query("Length > 7 and width < 8")
df.query("Name.str.startswith('abc')", engine="python")
```

**Subset Rows and Columns**

**Use df.loc[] and df.iloc[] to select only rows, only columns or both.**

**Use df.at[] and df.iat[] to access a single value by row and column.**

**First index selects row, second index selects column.**

**df.iloc[10:20]**  
Select rows 10-20.

**df.iloc[:, [1, 2, 3]]**  
Select columns in positions 1, 2 and 3 (first column is 0).

**df.loc[:, "a"]**  
Select all columns between a[] and b[] (inclusive).

**df.loc[df.a > 10, "a"]**  
Select rows meeting logical condition, and only the specific columns.

**df.iat[1, 2]**  
Access single value by index.

**df.iat[1, 2]**  
Access single value by index.

**Logic in Python (and pandas)**

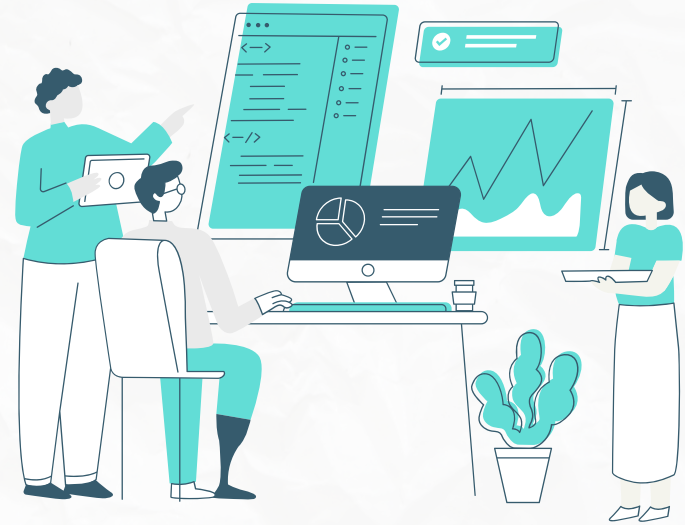
| Logic in Python (and pandas)        | Not equal to             |
|-------------------------------------|--------------------------|
| Less than                           | <                        |
| Greater than                        | >                        |
| Less than or equal to               | <=                       |
| Greater than or equal to            | >=                       |
| Not equal to                        | !=                       |
| Equal to                            | ==                       |
| Not None                            | is not None              |
| Logical and, or, not, xor, any, all | &,  , ~, ^, any(), all() |

Created for pandas by [Quansight Labs](#), originally written by [P. V. K.](#), updated by [Quansight Labs](#), licensed by [Quansight Labs](#).

# Ejemplos prácticos en Jupyter



# Descripción del trabajo y exposición final



# Armado de grupos para el trabajo final



Planilla con los grupos

Plazo para tener definidos los grupos de trabajo: **jueves 04/09/25**

# ¿Te quedaron dudas?



- Recordá que podés consultarnos durante la clase preniendo la cámara y hablando, o simplemente escribiendo en el chat.
- También podés escribirnos por correo las veces que sea necesario.



Esp. Lic. María Carina Roldán  
[macroldan@fi.uba.ar](mailto:macroldan@fi.uba.ar)



Esp. Ing. Ariadna Garmendia  
[arigarmendia@gmail.com](mailto:arigarmendia@gmail.com)

Y si tenés un feedback, un pedido, una sugerencia, etc. y no te animás a decirlo, podés dejarlo por escrito en la encuesta voluntaria y anónima (disponible durante todo el bimestre).